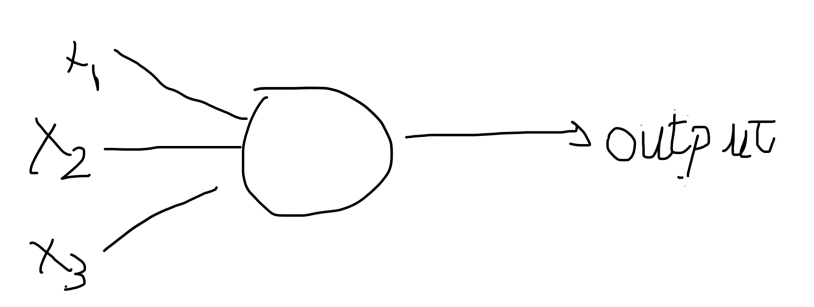
1 perceptrons

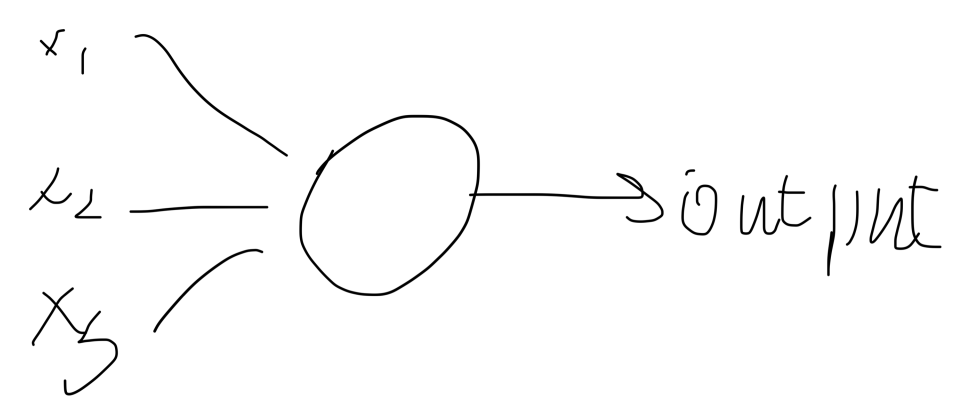

x_1
x_2
x_3
output

$$\text{output} = \begin{cases} 0 & \text{if } \sum_j w_j x_j \le \text{threshold} \\ 1 & \text{if } \sum_j w_j x_j > \text{threshold} \end{cases}$$

b = bias = − threshold

$$\text{output} = \begin{cases} 0 & \text{if } wx + b \le 0 \\ 1 & \text{if } wx + b > 0 \end{cases}$$

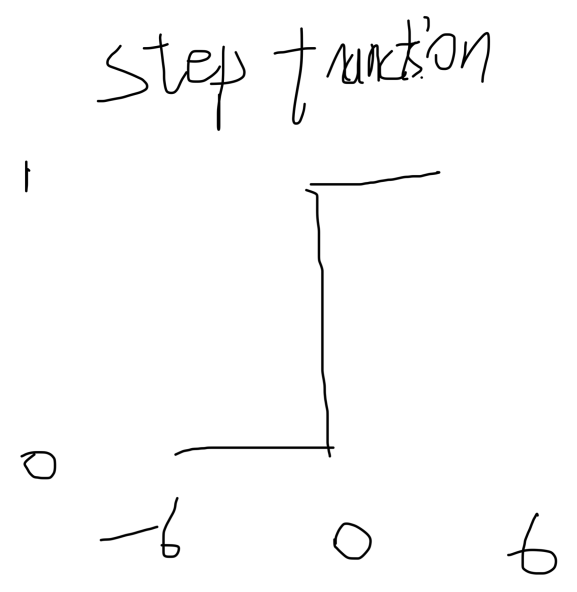
2 sigmoid notation

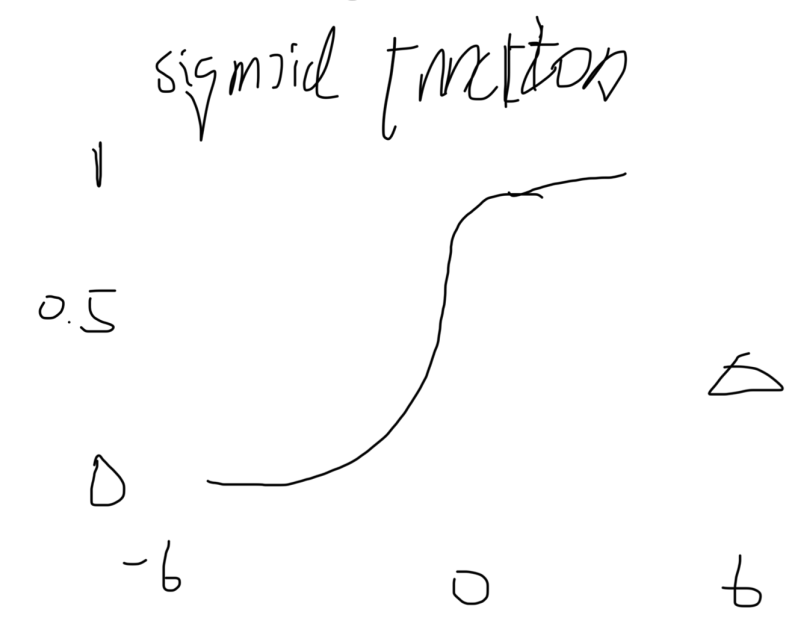

x_1
x_2
x_3
output

$z = \sum wx + b$

$\sigma(z) = \frac{1}{1+e^{-z}}$

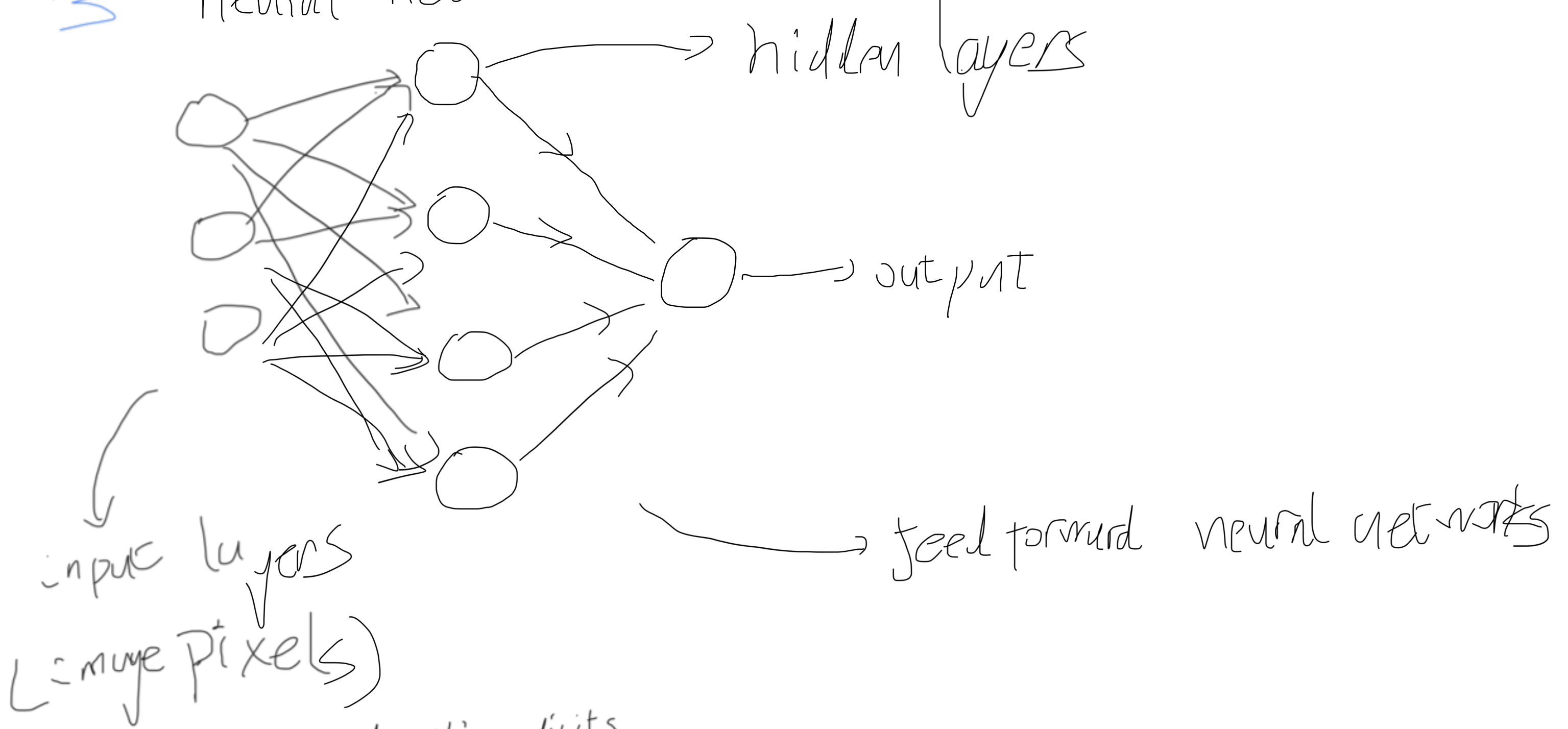
$\text{output} = \frac{1}{1 + \exp(-\sum w_j x_j - b)}$

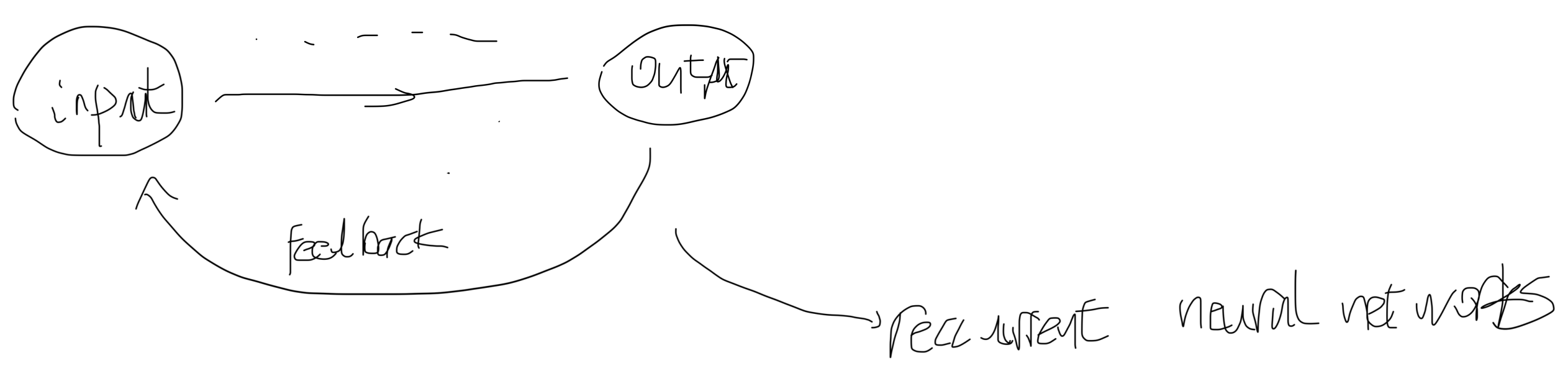




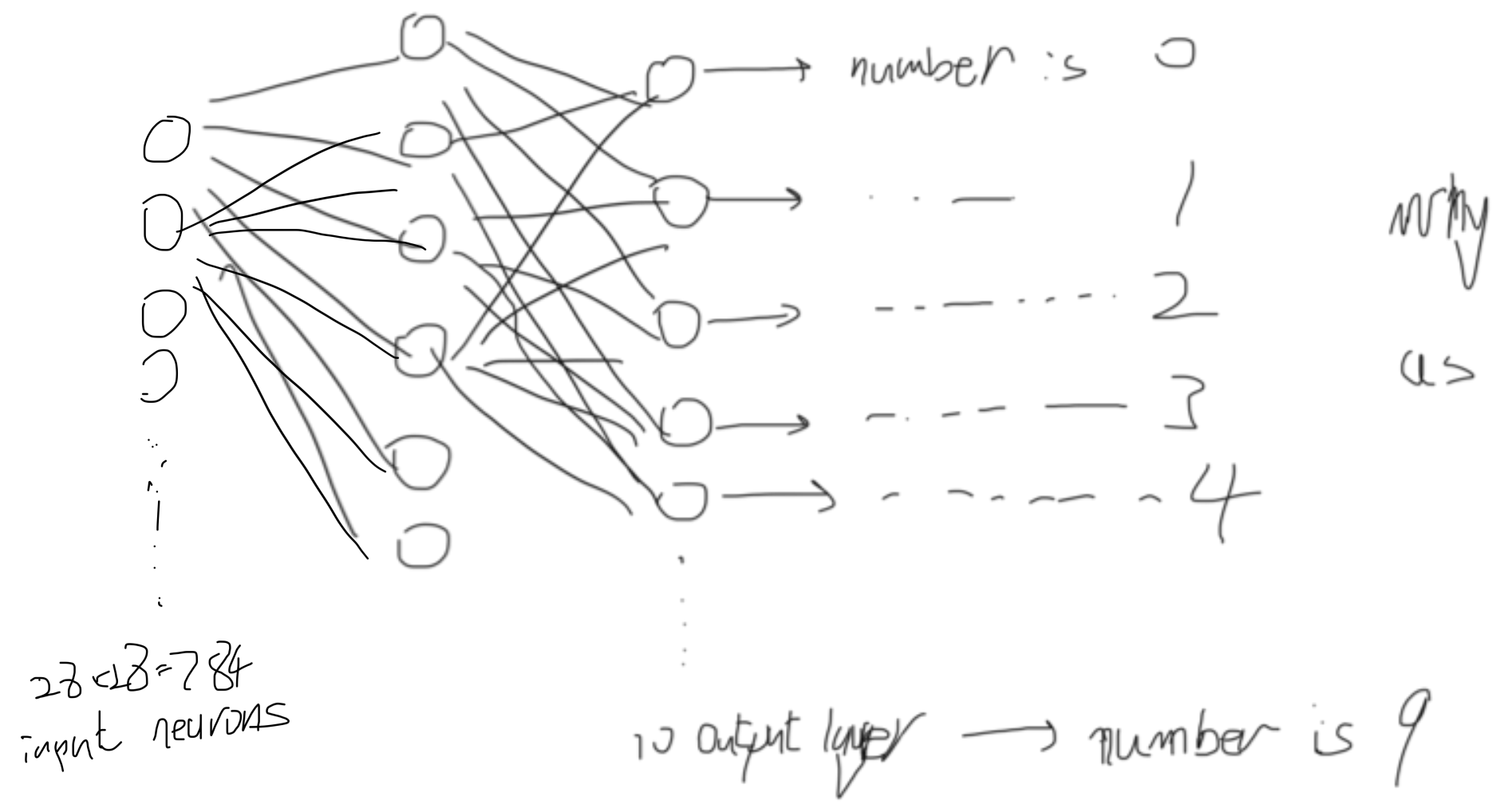
$\Delta \text{output} = \sum_j \frac{\partial \text{output}}{\partial w_j} \Delta w_j + \sum_j \frac{\partial \text{output}}{\partial b_j} \Delta b_j$

3 neural networks


hidden layers
output
input layers
(image pixels)
feed forward neural networks


input
output
feedback
recurrent neural networks

4 classifying handwritting digits


number is 0
1
2
3
4
28×28=784
input neurons
10 output layer → number is 9

why not use 4 output as $2^4 = 16$ is enough? → cant relate 4 numbers together

5. Learning with gradient descent

output = $y(x)$ if output = 6 $y(x) = (0\,0\,0\,0\,0\,0\,1\,0\,0\,0)^T$

cost function $C(w,b) = \frac{1}{2n}\sum_x \|y(x) - a\|^2$

$a \leftarrow x, w, b$

gradient descent → minimize $C(w,b)$

$\Delta C = \frac{\partial C}{\partial v_1}\Delta v_1 + \frac{\partial C}{\partial v_2}\Delta v_2$

$\Delta v = (\Delta v_1, \Delta v_2)^T$

gradient of C $\nabla C = \left(\frac{\partial C}{\partial v_1}, \frac{\partial C}{\partial v_2}\right)^T$

$\Delta C = \nabla C \cdot \Delta v$ let $\Delta v = -\eta \nabla C$

$\Delta C = -\eta \|\nabla C\|^2$

so $\Delta C \le 0$

$v' = v - \eta \nabla C$ let $\Delta v = \epsilon$

so $\eta = \epsilon / \|\nabla C\|$

5.2 learning with gradient descent

in neural networks $b' = b - \eta\frac{\partial C}{\partial b}$

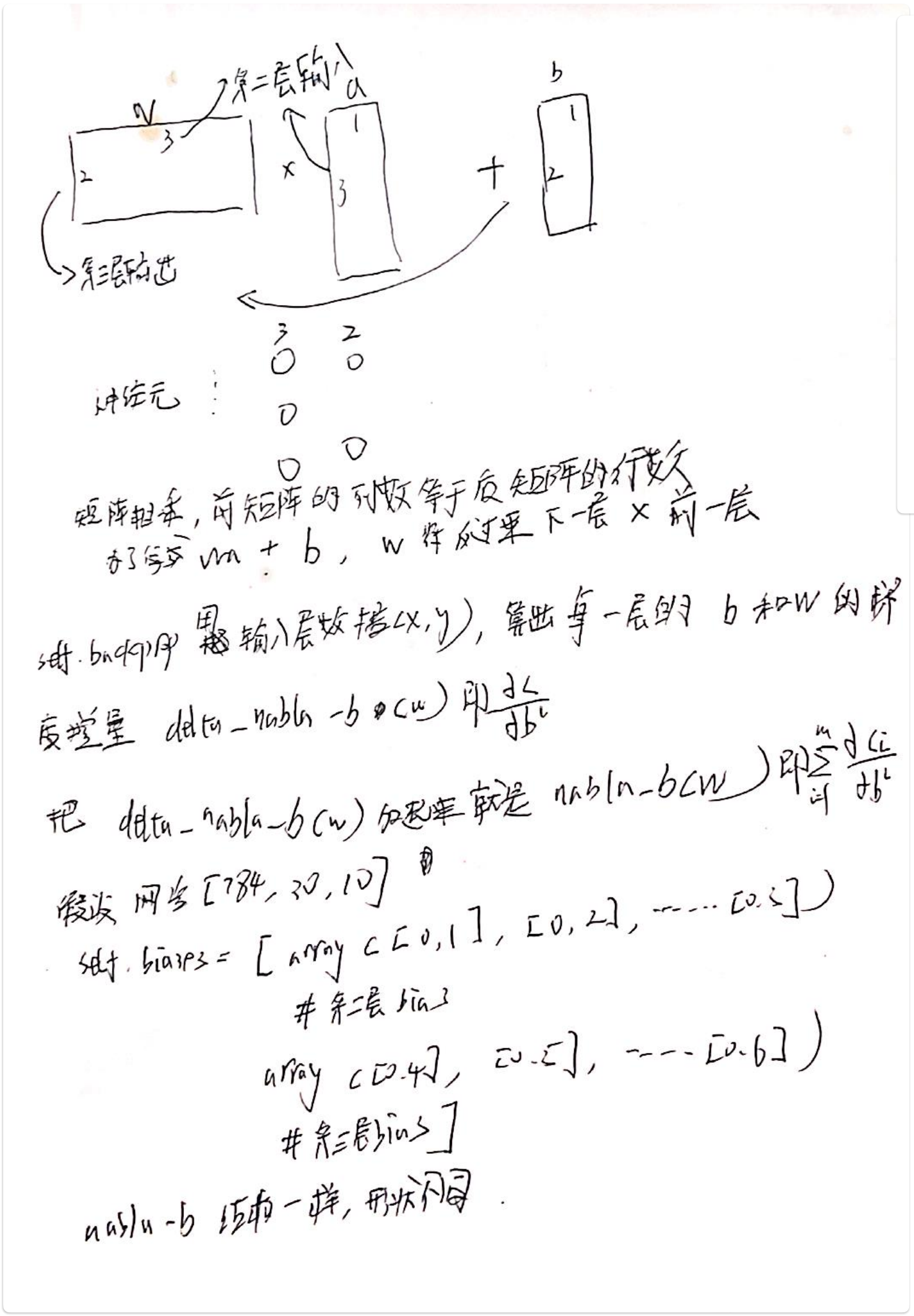
$w' = w - \eta\frac{\partial C}{\partial w}$

stachastic gradient descent $\frac{\sum_{j=1}^{m} \nabla C_{x_j}}{m} \approx \frac{\sum_x \nabla C_x}{n} = \nabla C$

$w' = w - \frac{\eta}{m}\sum_j \frac{\partial C_{x_j}}{\partial w}$

$b' = b - \frac{\eta}{m}\sum_j \frac{\partial C_{x_j}}{\partial b}$

6


w
3
2
第二层输入
a
1
3
b
1
2
第三层输出
3
2
神经元

矩阵相乘，前矩阵的列数等于后矩阵的行数

为了写成 $wa + b$，w 得对乘 下一层 × 前一层

self.backprop(用输入层数据(x, y))，算出每一层的 b 和 w 的梯

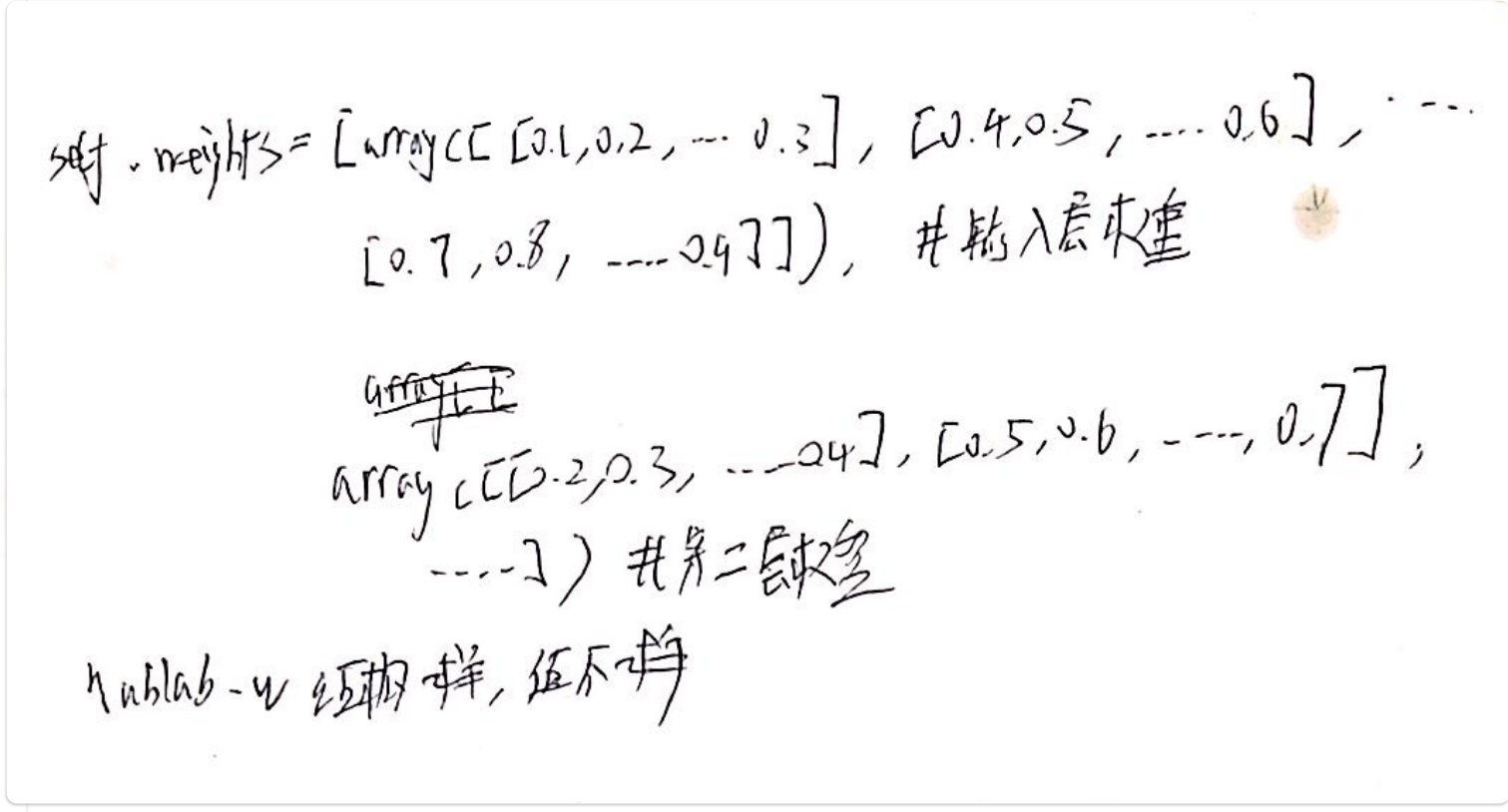
度变量 delta_nabla_b(w) 即 $\frac{\partial C}{\partial b^l}$

把 delta_nabla_b(w) 加起来就是 nabla_b(w) 即 $\sum_{i=1}^{m}\frac{\partial C_i}{\partial b^l}$

假设 网络 [784, 30, 10]

```
self.biases = [array([0,1], [0,2], ...... [0.3])
               # 第二层 bias
               array([0.4], [0.5], ---- [0.6])
               # 第三层 bias]
```

nabla_b 结构一样，形状相同



```
self.weights = [array([[0.1, 0.2, ... 0.3], [0.4, 0.5, .... 0.6], ...
                [0.7, 0.8, .... 0.9]]), # 输入层权重
                ~~array([[~~
                array([[0.2, 0.3, ...0.4], [0.5, 0.6, ---, 0.7],
                  ....]) # 第二层权重
```

nabla_w 结构一样，值不同